Linux网络--Http静态资源服务器

- 1. 静态资源服务器的职责
  - 1.1 Web 根目录
  - 1.2 路径规范化与安全
  - 1.3 同一资源的缓存
- 2. 文件读取
  - 2.1 文本读取为什么不够
  - 2.2 C++ 二进制读取
    - 2.2.1 获取文件大小
    - 2.2.2 为什么使用 vector
- 3. Content-Type 与 MIME
  - 3.1 用 Content-Type 告诉浏览器文件类型
  - 3.2 后缀提取
  - 3.3 Content-Type 与 Content-Encoding
- 4. 构造静态响应
  - 4.1 状态码与 Header
  - 4.2 Content-Length
  - 4.3 Header 序列化
  - 4.4 404 与 favicon
- 5. 静态服务器处理流程
  - 5.1 常见故障

1. 静态资源服务器的职责

静态资源服务器负责把 URL 路径映射到 Web 根目录中的文件，读取文件内容，构造成 HTTP 响应并发送给客户端。

HTML、CSS、JavaScript、图片、音频、视频、字体和下载文件都可以是静态资源：

```
/index.html
/css/app.css
/js/app.js
/image/cover.png
/video/demo.mp4
```

静态资源的本质都是字节流。浏览器之所以能区分网页、图片和视频，主要依靠响应中的 `Content-Type`。

上一篇介绍了完整请求和响应模型：Linux网络--Http请求与响应

1.1 Web 根目录

URL 中的 `/` 是站点外部根目录，不是 Linux 文件系统根目录。服务器会把请求路径拼接到配置的 Web 根目录：

```
Web 根目录：/srv/wwwroot
请求目标：/image/logo.png
最终文件：/srv/wwwroot/image/logo.png
```

典型目录结构：

```
wwwroot/
  css/
    app.css
  image/
    cover.png
  js/
    app.js
  video/
    demo.mp4
  index.html
  404.html
```

请求 `/` 时通常自动映射到默认首页：

```
/
  -> /index.html
  -> /srv/wwwroot/index.html
```

如果站点提供多个默认文件，可以按配置顺序检查，例如 `index.html`、`index.htm`。

1.2 路径规范化与安全

服务器不能直接把用户输入拼接成文件系统路径：

```
用户路径：../../etc/passwd
危险结果：/srv/wwwroot/../../etc/passwd
```

安全处理至少应包含：

1. 限制原始请求路径长度。
2. 解析百分号编码。
3. 拒绝控制字符和非法路径。
4. 规范化 `.`、`..` 和连续斜杠。
5. 将结果转换为绝对路径。
6. 确认最终路径仍位于 Web 根目录内。
7. 按业务需要禁止符号链接越过根目录。
8. 禁止访问隐藏文件、配置文件和版本控制目录。

只检查字符串中是否包含 `..` 不够，因为 URL 编码和符号链接都可能改变最终文件位置。

1.3 同一资源的缓存

浏览器会缓存图片、CSS、JavaScript 等静态资源。页面多次引用同一 URL 时，通常只产生一次网络请求，再在多个位置复用。

常见缓存 Header：

Header	作用
`Cache-Control`	控制缓存范围和时长
`ETag`	表示资源版本
`Last-Modified`	表示最后修改时间
`Expires`	指定绝对过期时间

客户端再次验证时，可以发送：

```
If-None-Match: "abc123"
If-Modified-Since: Wed, 21 Oct 2026 07:28:00 GMT
```

资源没有变化时，服务器返回 304，不重复发送正文：

```
HTTP/1.1 304 Not Modified
ETag: "abc123"
```

2. 文件读取

2.1 文本读取为什么不够

图片、视频和压缩文件都可能包含 `\0`、换行字节或其他特殊序列。按文本行读取可能在第一个特殊字节处提前结束，最终只返回文件的一部分。

文件读取应始终能处理任意字节，因此应使用二进制模式。

2.2 C++ 二进制读取

```cpp
#include <fstream>
#include <optional>
#include <string>

std::optional<std::string> readFileBinary(const std::string& path) {
    std::ifstream in(path, std::ios::binary);
    if (!in) {
        return std::nullopt;
    }

    in.seekg(0, std::ios::end);
    const std::streampos end = in.tellg();
    if (end < 0) {
        return std::nullopt;
    }

    in.seekg(0, std::ios::beg);
    std::string content(static_cast<std::size_t>(end), '\0');
    in.read(content.data(), static_cast<std::streamsize>(content.size()));
    if (in.gcount() != static_cast<std::streamsize>(content.size())) {
        return std::nullopt;
    }
    return content;
}
```

关键点：

- 使用 `std::ios::binary`。
- 使用 `seekg` 和 `tellg` 获取文件大小。
- 使用 `content.data()` 作为写入缓冲区。
- 检查实际读取数量，不能只判断流是否打开。
- 空文件也是一种合法文件，读取结果可以是空字符串。

2.2.1 获取文件大小

C++ 文件流方式：

```
in.seekg(0, std::ios::end);
auto size = in.tellg();
in.seekg(0, std::ios::beg);
```

系统调用方式：

```
struct stat st;
if (stat(path.c_str(), &st) == 0) {
    auto size = st.st_size;
}
```

`stat` 还可以获取 inode、权限、所有者、硬链接数和修改时间。

2.2.2 为什么使用 vector

也可以用 `std::vector<char>` 保存二进制数据：

```
std::vector<char> content(fileSize);
in.read(content.data(), fileSize);
```

`vector<char>` 更强调内容是任意字节；`std::string` 也支持二进制数据，但业务代码必须避免把它误当成以 `\0` 结尾的 C 字符串。

3. Content-Type 与 MIME

3.1 用 Content-Type 告诉浏览器文件类型

```
Content-Type: text/html; charset=utf-8
Content-Type: text/css; charset=utf-8
Content-Type: text/javascript; charset=utf-8
Content-Type: image/png
Content-Type: video/mp4
```

服务器通常根据文件后缀查表：

后缀	Content-Type
`.html`	`text/html; charset=utf-8`
`.htm`	`text/html; charset=utf-8`
`.css`	`text/css; charset=utf-8`
`.js`	`text/javascript; charset=utf-8`
`.json`	`application/json; charset=utf-8`
`.xml`	`application/xml; charset=utf-8`
`.png`	`image/png`
`.jpg`、`.jpeg`	`image/jpeg`
`.gif`	`image/gif`
`.svg`	`image/svg+xml`
`.mp3`	`audio/mpeg`
`.mp4`	`video/mp4`
`.pdf`	`application/pdf`
未知类型	`application/octet-stream`

不要把所有未知后缀都标成 HTML。错误类型可能导致渲染异常或安全风险。

3.2 后缀提取

从 URI 的路径部分提取最后一个点之后的后缀：

```
std::string suffix = ".html";
const auto slash = uri.rfind('/');
const auto dot = uri.rfind('.');
if (dot != std::string::npos &&
    (slash == std::string::npos || dot > slash)) {
    suffix = uri.substr(dot);
}
```

提取前应先把查询字符串从 URI 中分离，否则 `style.css?v=1` 可能被错误解析。

3.3 Content-Type 与 Content-Encoding

`Content-Type` 描述正文媒体类型，`Content-Encoding` 描述传输前的压缩方式：

```
Content-Type: text/css; charset=utf-8
Content-Encoding: gzip
```

浏览器需要先按 `Content-Encoding` 解压，再按 `Content-Type` 解释内容。

4. 构造静态响应

4.1 状态码与 Header

文件存在时：

```
HTTP/1.1 200 OK
Content-Type: image/png
Content-Length: 91027
Connection: keep-alive

[二进制正文]
```

文件不存在时：

```
HTTP/1.1 404 Not Found
Content-Type: text/html; charset=utf-8
Content-Length: 3099
Connection: close

[404 页面]
```

`404` 表示请求的资源不存在，错误页面则向用户解释情况。状态码和正文都重要。

4.2 Content-Length

静态文件通常可以提前得到长度，因此使用：

```
Content-Length: body.size()
```

注意这是字节数。响应正文是二进制字符串时，`std::string::size()` 返回的正是字节数。

如果正文由动态流生成，无法提前确定长度，可以使用 HTTP/1.1 的 `Transfer-Encoding: chunked`。具体分块规则见 Linux网络--Http请求与响应 > 1.7 正文边界与报文读取。

4.3 Header 序列化

响应头通常保存在键值表中：

```
HTTP/1.1 200 OK\r\n
Content-Type: image/png\r\n
Content-Length: 91027\r\n
Connection: keep-alive\r\n
\r\n
[正文]
```

HTTP/1.x 每行以 `\r\n` 结束，最后还需一个空行。发送时不能只调用一次 `write()` 就认为全部字节已经发出，应循环处理部分写入。

4.4 404 与 favicon

浏览器可能请求 `/favicon.ico`。服务器应返回一个小图标，或者明确返回 404；不应让这个辅助请求无限挂起。

自定义 404 页面适合放在 Web 根目录：

```
/srv/wwwroot/404.html
```

处理流程：

1. 目标文件不存在。
2. 设置状态码 404。
3. 读取 404 页面。
4. 设置 `Content-Type: text/html`。
5. 设置 `Content-Length`。

6. 序列化完整响应。

5. 静态服务器处理流程

```
accept 客户端连接
  -> 读取完整请求 Header
  -> 若是 POST，则继续读取正文
  -> 解析方法和 URI
  -> 分离 URI 的路径和查询字符串
  -> 规范化路径
  -> 拼接 Web 根目录
  -> 检查路径没有越界
  -> 以二进制方式读取文件
  -> 根据后缀选择 Content-Type
  -> 设置 Content-Length
  -> 构建并序列化响应
  -> 循环发送全部字节
  -> 根据 Connection 继续处理或关闭
```

5.1 常见故障

现象	可能原因
首页正常，图片不显示	图片按文本读取、Content-Type 错误、正文截断
浏览器一直等待	Content-Length 错误、连接声明与关闭行为不一致
页面 404	Web 根目录配置错误、默认首页不存在、路径规范化错误
可以请求文件，但浏览器下载而不是展示	Content-Type 缺失或被设成 `application/octet-stream`
CSS 或 JS 不生效	后缀、Content-Type、缓存或响应状态错误
访问不存在的路径返回 200	错误处理逻辑把 404 页面错误地当成成功响应

下一篇进入动态网站，处理表单、GET/POST、参数解析和 URI 路由：Linux网络--Http表单与动态路由